

Fruit and Vegetable Image Classification Report

Group: 5130203/20102

Team Members:

Petrov Nikita	Project Manager
Tsvirkun Nikita	ML Engineer
Nikolaev Alexander	Frontend
Maximov Ivan	Backend

December 25, 2024

1 Problem Definition

The objective of this project is to develop a deep learning model for the classification of images into distinct categories of fruits and vegetables. This task falls under the domain of computer vision, with a focus on image recognition and classification. The ultimate goal is to accurately identify the type of fruit or vegetable depicted in a given image.

1.1 Dataset

The dataset used for this project is a comprehensive collection of images featuring various fruits and vegetables. It is structured into three subsets: Train, Test, and Validation. The dataset details are as follows:

- **Fruits:** Banana, Apple, Pear, Grapes, Orange, Kiwi, Watermelon, Pomegranate, Pineapple, Mango.
- **Vegetables:** Cucumber, Carrot, Capsicum, Onion, Potato, Lemon, Tomato, Radish, Beetroot, Cabbage, Lettuce, Spinach, Soybean, Cauliflower, Bell Pepper, Chilli Pepper, Turnip, Corn, Sweetcorn, Sweet Potato, Paprika, Jalapeño, Ginger, Garlic, Peas, Eggplant.

1.2 Structure of the Dataset

The dataset is organized into three main folders:

- **Train:** Contains 100 images per category.
- **Test:** Contains 10 images per category.
- **Validation:** Contains 10 images per category.

Each folder is subdivided into specific folders for each type of fruit and vegetable, containing the respective images.

1.3 Classes

The dataset comprises a total of 36 distinct classes, representing a wide range of fruits and vegetables. Each class corresponds to a specific type of fruit or vegetable, making the classification task a multi-class classification problem.

1.4 Goal

The primary goal is to train a convolutional neural network (CNN) to effectively classify images into these 36 categories. The trained model will facilitate accurate identification of fruits and vegetables from images, which can be leveraged for various applications, such as recipe suggestions or dietary analysis.

2 Methods and Approach

2.1 Tools and Infrastructure

The model was implemented using the following tools and infrastructure:

- **PyTorch Version:** 2.5.1+cu118
- **CUDA Version:** 11.8
- **GPU Used:** NVIDIA GeForce RTX 2070 SUPER
- **Device:** CUDA (GPU)
- **Training Environment:** Single GPU setup with PyTorch

2.2 Model Architecture

In this project, two different architectures were used: a custom CNN model (FruitVegCNN) and a modified ResNet50 (ResNet50Custom). Both models were used to classify the 36 categories of fruits and vegetables.

2.2.1 FruitVegCNN Architecture

The FruitVegCNN model is a convolutional neural network with five convolutional blocks and fully connected layers. The network performs the following operations:

- Convolutional layers with kernel size 3x3, ReLU activation, batch normalization, and max-pooling.
- Fully connected layers with dropout for regularization.
- Final output layer with softmax activation to predict one of 36 classes.
- The model contains about 9 million training parameters.

2.2.2 ResNet50Custom Architecture

The ResNet50Custom model utilizes the pre-trained ResNet50 architecture as the backbone, with the final classification layer replaced by a custom fully connected network. Key aspects:

- Pre-trained layers are frozen to leverage learned feature representations.
- The final layer consists of hidden layers (1024 and 512 neurons), with dropout and ReLU activations.
- The final output layer has 36 neurons, corresponding to the number of classes.
- The model contains about 3 million training parameters.

2.3 Training Configuration

- **Loss Function:** CrossEntropyLoss with label smoothing (0.1)
- **Optimizer:** AdamW with weight decay (L2 regularization)
- **Learning Rate Scheduler:** OneCycleLR for dynamic learning rate adjustment
- **Training Strategy:** Mixed Precision Training with AMP for faster training and lower memory usage

The training process uses the OneCycleLR scheduler to adjust the learning rate dynamically during the training process, ensuring better convergence. Mixed precision training with automatic casting and gradient scaling helps reduce memory usage and speed up computations.

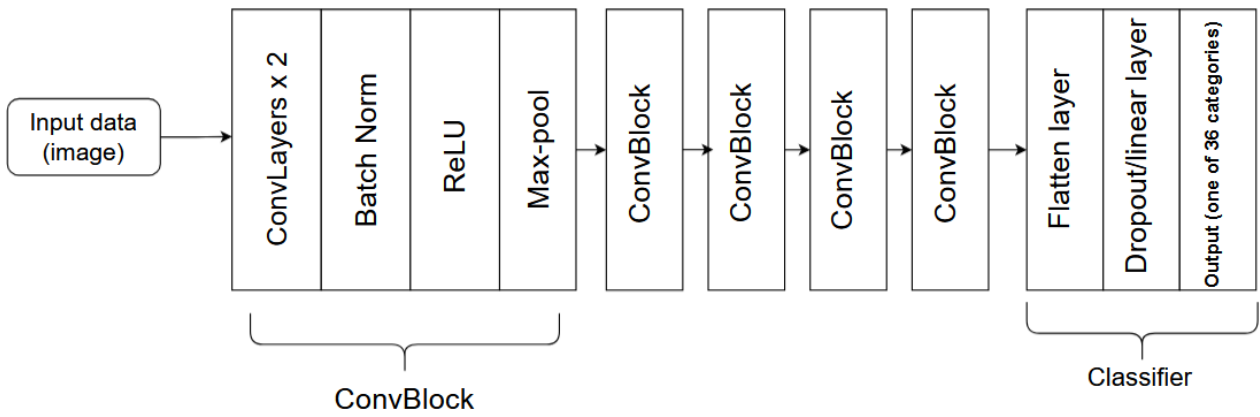


Figure 1: FruitVegCNN Architecture

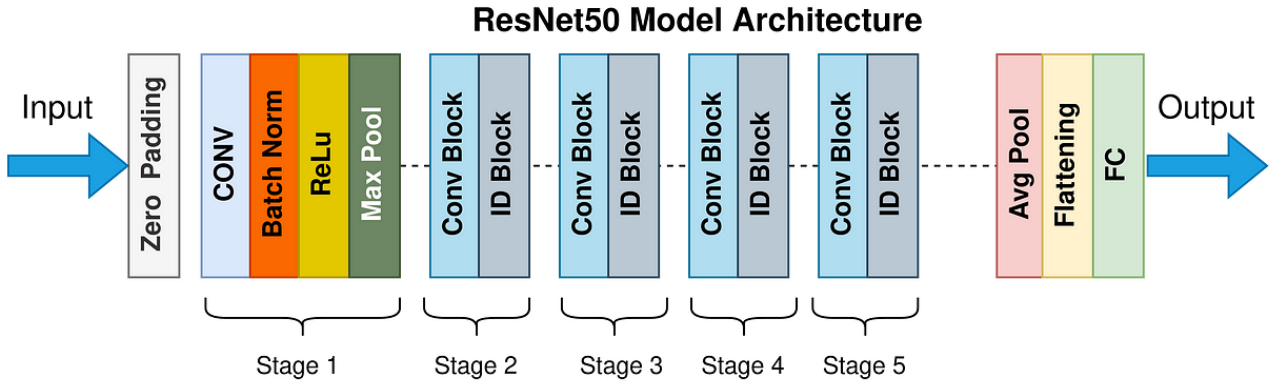


Figure 2: ResNet50Custom Architecture

3 Results: Comparison of Models with Different Configurations

In this section, we compare the performance of two models: FruitVegCNN and ResNet50Custom, both trained on the same dataset. Performance comparison includes training and validation loss/accuracy plots, as well as detailed evaluation metrics, such as accuracy, precision, recall, and F1 score.

3.1 FruitVegCNN Results

The training process of the FruitVegCNN model was carried out for 50 epochs. The model's performance was evaluated in terms of both training and validation loss, as well as accuracy. The following are the results for the testing phase:

Metric	Value
Accuracy	0.6908
Precision	0.7139
Recall	0.6908
F1-Score	0.6788

The training and validation loss/accuracy graphs for the FruitVegCNN model are shown below:



Figure 3: Loss/Accuracy graphs for FruitVegCNN Model

3.2 ResNet50Custom Results

The second model, ResNet50Custom, was trained using the OneCycleLR scheduler and mixed-precision training for improved efficiency. This model was also trained for 50 epochs. The performance metrics for the ResNet50Custom model are as follows:

Metric	Value
Accuracy	0.9582
Precision	0.9638
Recall	0.9582
F1-Score	0.9576

The training and validation loss/accuracy graphs for the ResNet50Custom model are shown below:

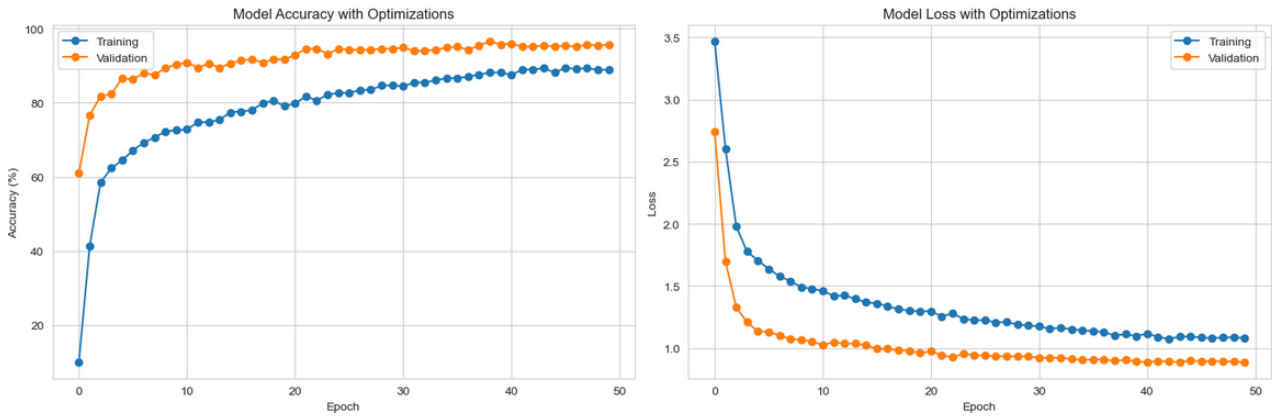


Figure 4: Loss/Accuracy graphs for ResNet50 Model

3.3 Comparison of Models

The two models exhibit vastly different performances. The ResNet50Custom model significantly outperforms the FruitVegCNN model in terms of accuracy, precision, recall, and F1-score. Below is a summary of the key performance metrics for both models:

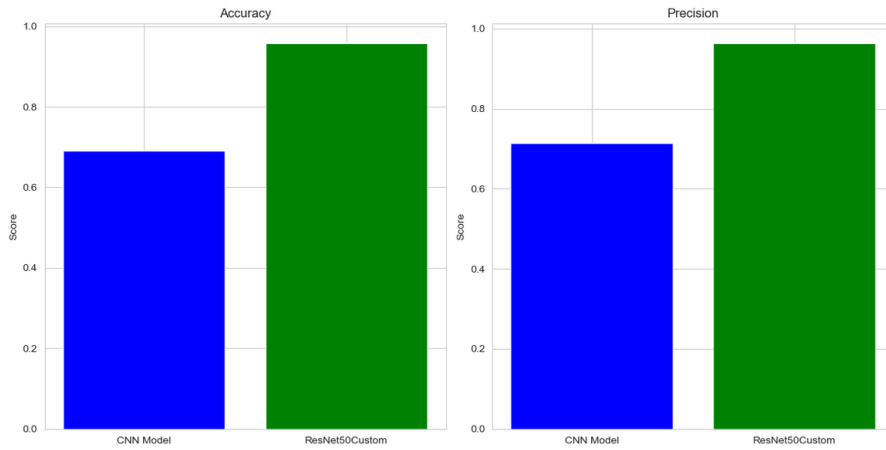


Figure 5: Accuracy/Precision comparison graphs

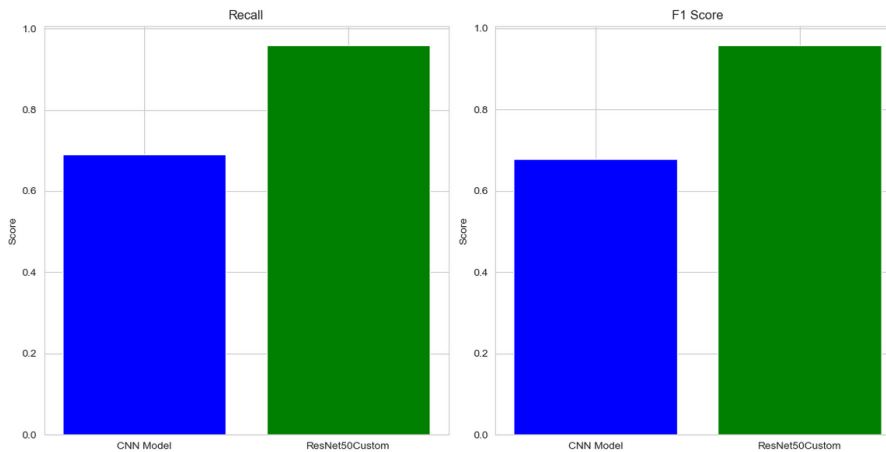


Figure 6: Recall/F1-score comparison graphs

4 Conclusion

In this study, two models, FruitVegCNN and ResNet50Custom, were developed and evaluated for the task of fruit and vegetable image classification. Both models demonstrated their strengths and limitations under different conditions, leading to the following conclusions:

4.1 Strengths of ResNet50Custom

The primary strengths of ResNet50Custom include:

- **Robust Feature Extraction:** The pre-trained ResNet50 backbone provides a strong starting point, reducing the burden of learning from scratch.
- **Enhanced Generalization:** Dropout layers and batch normalization in its custom classifier contribute to better regularization, mitigating overfitting.
- **Efficient Training:** Techniques like learning rate scheduling and gradient scaling ensured smooth convergence.

4.2 Limitations and Challenges

Despite its impressive performance, ResNet50Custom has the following limitations:

- **High Computational Cost:** The model's large size and complexity result in significant memory and computational demands, making it less suitable for deployment on resource-constrained devices.
- **Overfitting Risk:** Although regularization techniques were employed, fine-tuning pre-trained models on small datasets can still lead to overfitting.

Susceptibility to Gradient-Based Attacks

Another critical limitation is the vulnerability of ResNet50Custom to adversarial attacks. Gradient-based adversarial methods, such as FGSM (Fast Gradient Sign Method) or PGD (Projected Gradient Descent), can subtly manipulate input images to deceive the model. This susceptibility arises because ResNet50Custom, like most deep neural networks, heavily relies on gradient-based optimization, making it sensitive to adversarial perturbations.

Mitigating these vulnerabilities requires:

- **Adversarial Training:** Incorporating adversarially perturbed examples during training can enhance robustness.
- **Regularization Techniques:** Approaches like weight pruning, gradient masking, and spectral normalization can improve resilience against such attacks.
- **Model Uncertainty Estimation:** Bayesian methods can help quantify uncertainty, enabling the model to identify potentially adversarial samples.

4.3 Future Directions

To further improve model performance and robustness, future work could focus on:

- Expanding the dataset to include more diverse and representative images.

- Exploring ensemble learning by combining predictions from multiple models for enhanced accuracy.
- Developing lightweight model architectures tailored for real-time deployment.
- Incorporating adversarial defenses to ensure robustness in security-critical applications.

In conclusion, ResNet50Custom stands out as the optimal choice for the current task due to its high accuracy and balanced performance metrics. However, addressing its limitations and susceptibility to adversarial attacks will be essential for achieving robust and practical deployment.